

STARIAN CHECKLIST

Relatório Técnico de Refatoração Full Stack

Angular 17 | Laravel 11 | SQLite | Docker

Candidato: não informado no repositório
22 de julho de 2026

Desenvolvido com apoio de Inteligência Artificial,
com revisão e validação humana.

Sumário

- 1 Resumo executivo
- 2 Contexto e diagnóstico inicial
- 3 Objetivos da refatoração
- 4 Estratégia de execução
- 5 Principais decisões arquiteturais
- 6 Melhorias no backend
- 7 Melhorias no frontend
- 8 Estratégia de testes
- 9 Uso de Inteligência Artificial
- 10 Dosagem de IA
- 11 Impacto técnico e de negócio
- 12 Indicadores de sucesso
- 13 Resultados finais
- 14 Limitações e melhorias futuras
- 15 Conclusão

1. Resumo executivo

O projeto original era uma checklist funcional, porém construída como um exemplo intencional de dívida técnica. O Angular concentrava comunicação HTTP, estado, template e apresentação no componente raiz. O Laravel executava regras e persistência JSON diretamente nas rotas e as carregava pelo middleware `web`. Erros eram mascarados por dados fictícios ou falsos sucessos; entradas inválidas eram aceitas; não havia garantia de integridade para títulos duplicados; e o Docker não reproduzia de forma confiável o ambiente.

A estratégia adotada foi incremental. Primeiro foi estabelecido um baseline executável; depois foram corrigidos ambiente, TLS e debug. Testes de contrato registraram o comportamento válido antes da mudança de persistência. Em seguida o backend foi migrado para uma API stateless com SQLite/Eloquent, o frontend foi componentizado e os fluxos críticos receberam estados e validações explícitas. Por fim, foi adicionada proteção contra duplicidade, um runner Angular com Chromium isolado e uma auditoria completa.

O resultado final preserva as três funcionalidades do escopo - listar, criar e remover - com responsabilidades separadas, contrato HTTP previsível e execução reproduzível. A validação final registrou:

- backend: 28 testes aprovados, 106 assertions, taxa de aprovação de 100%;
- frontend: 24 specs aprovados no Chromium, taxa de aprovação de 100%;
- cobertura Angular: 93,90% statements, 80,95% branches, 100% functions e 93,58% lines;
- migrations, rollback, seed, Pint, TypeScript e build de produção aprovados;
- integração HTTP validada com `200`, `201`, `422`, `204` e `404`;
- ambientes Docker normal, debug e testes construídos e validados.

A qualidade final não depende de abstrações excessivas. A solução utiliza os recursos nativos de Angular e Laravel necessários para o tamanho do domínio e mantém explícitos os trade-offs para uma evolução futura.

2. Contexto e diagnóstico inicial

2.1 Funcionamento original

A aplicação expunha uma checklist simples. O frontend buscava tarefas, permitia inserir um título e remover um item. O backend gravava uma coleção em `storage/tarefas.json`. Na prática, o comportamento nominal existia, mas o desenho interno não protegia o usuário nem o processo de manutenção.

No Angular, o `AppComponent` acumulava a lista, as chamadas HTTP, as regras de erro, o formulário e toda a apresentação. Tipos `any` eliminavam parte da segurança estática. A URL `http://localhost:8000/tarefas` estava acoplada ao componente. Em falhas de API, a interface criava dados locais ou sinalizava sucesso, produzindo uma divergência entre o que o usuário via e o que havia sido persistido.

No Laravel, funções globais em `routes/api.php` manipulavam JSON diretamente. O arquivo era incluído por `routes/web.php`, levando as rotas para o stack de sessão e CSRF. Ausência de título gerava “Tarefa sem título”; strings vazias, arrays e textos acima de 255 caracteres não eram rejeitados; e uma exclusão inexistente retornava `204`, como se tivesse ocorrido.

2.2 Riscos técnicos

A persistência JSON não oferecia as garantias naturais de uma base transacional. Escritas concorrentes poderiam sobrescrever conteúdo, a geração de IDs dependia de manipulação manual e não existia constraint de unicidade. A lógica em rotas dificultava teste, reutilização e evolução.

O frontend monolítico criava forte acoplamento: uma alteração visual podia afetar comunicação e estado. O uso de `any` adiava erros para runtime. Subscriptions e estados implícitos dificultavam raciocinar sobre carregamento, envio ou remoção simultânea.

O ambiente Docker possuía bind mounts em diretórios diferentes dos `WORKDIR`, ocultava `vendor` e `node_modules`, não incluía SQLite no PHP e usava uma instalação npm não reproduzível. Isso elevava o custo de onboarding e tornava diagnósticos dependentes da máquina de cada pessoa.

2.3 Impactos para usuário, manutenção e negócio

- Falso sucesso: quando a interface afirma que uma criação ou remoção ocorreu sem confirmação do servidor, o usuário perde confiança e pode tomar decisões sobre dados inexistentes.
- Integridade frágil: o JSON aumenta risco de perda ou corrupção de dados e dificulta concorrência segura.
- Mudança cara: ausência de testes e responsabilidades misturadas aumentam o esforço de cada alteração e o risco de regressão.
- Onboarding lento: Docker inconsistente consome tempo antes de qualquer contribuição funcional.
- Experiência limitada: baixa responsividade e feedback impreciso prejudicam uso mobile e compreensão de falhas.
- Acessibilidade insuficiente: ausência de semântica e relações ARIA reduz a efetividade para teclado e tecnologias assistivas.
- Contrato ambíguo: códigos incorretos e formatos instáveis ampliam custo de integração entre frontend e backend.

3. Objetivos da refatoração

A refatoração foi guiada pelos seguintes objetivos:

- 1 preservar o comportamento válido consumido pelo frontend;
- 2 corrigir entradas inválidas, remoção inexistente e falsos sucessos;

- 3 separar apresentação, comunicação, validação e persistência;
- 4 aumentar confiabilidade sem ampliar o produto;
- 5 melhorar legibilidade, responsividade e acessibilidade;
- 6 tornar instalação, debug, testes e build reproduzíveis;
- 7 criar uma rede de segurança automatizada antes das mudanças estruturais;
- 8 proteger a unicidade mesmo sob requisições simultâneas;
- 9 manter TLS seguro no ambiente com inspeção HTTPS;
- 10 evitar overengineering.

O critério de proporcionalidade foi central. Não foram adicionados autenticação, paginação, edição, conclusão de tarefas, NgRx, repository pattern, serviços de domínio ou infraestrutura externa sem necessidade.

4. Estratégia de execução

A execução foi dividida em dez etapas delimitadas:

- 1 Diagnóstico: leitura da estrutura e identificação de responsabilidades, riscos e contratos.
- 2 Baseline: execução do que existia para distinguir falhas preexistentes de regressões.
- 3 Ambiente: correção de `.env.example`, SQLite, Dockerfiles, volumes e instalações reproduzíveis.
- 4 Debug: Xdebug isolado, `ng serve`, source maps e configurações do VS Code.
- 5 Caracterização: testes HTTP sobre GET, POST e DELETE antes de trocar a persistência.
- 6 Backend: API stateless com controller, request, resource, model, migration e Eloquent.
- 7 Frontend: componentes pequenos, service tipado, Reactive Forms e estados explícitos.
- 8 Regra adicional: prevenção de duplicidade no cliente, servidor e banco.
- 9 Testes completos: Chromium isolado, suíte Angular real e cobertura.
- 10 Auditoria: higiene, validações finais e documentação de entrega.

Essa sequência reduziu risco porque cada etapa criou evidência para a seguinte. O contrato foi registrado antes da alteração de persistência; o backend foi estabilizado antes de mudar o cliente; a regressão do submit foi tratada antes de ampliar testes; e a constraint foi adicionada somente após a normalização estar definida. Assim, um problema podia ser atribuído a uma mudança específica, em vez de surgir dentro de uma refatoração ampla.

5. Principais decisões arquiteturais

5.1 JSON para SQLite/Eloquent

Problema. O arquivo JSON exigia controle manual de IDs, leitura e escrita integral e não oferecia constraint ou transação adequada.

Alternativas. Manter JSON com locking; usar SQLite; introduzir MySQL ou PostgreSQL.

Decisão. Adotar SQLite com Eloquent.

Justificativa. SQLite entrega schema, migrations, constraints e API Eloquent sem adicionar outro serviço ao Docker. É suficiente para uma checklist pequena e permite testes em memória.

Trade-off. SQLite possui limitações de concorrência e escala quando comparado a um servidor de banco. Caso o produto cresça, a abstração Eloquent facilita migrar a configuração sem reescrever o contrato HTTP.

Resultado. Persistência determinística, rollback validado, IDs do banco, casts e proteção de unicidade.

5.2 API stateless

Problema. As rotas de tarefas passavam pelo middleware `web`, levando sessão e CSRF a endpoints consumidos como API.

Alternativas. Manter `web` e contornar CSRF; adicionar prefixo `/api`; carregar `routes/api.php` sem prefixo.

Decisão. Registrar `routes/api.php` no stack `api` com `apiPrefix: ''`.

Justificativa. A decisão preserva os caminhos públicos `/tarefas`, remove dependência de sessão/CSRF e usa a configuração nativa do Laravel 11.

Trade-off. O contrato sem prefixo exige atenção para evitar colisões futuras com páginas web. No escopo atual há somente três rotas públicas.

Resultado. `route:list -v` confirma três rotas com middleware `api` e nenhuma com `web`.

5.3 Controller, Request e Resource

Problema. Rotas acumulavam validação, regra, persistência e serialização.

Alternativas. Manter closures; concentrar tudo no controller; separar com recursos nativos.

Decisão. Usar `TaskController`, `StoreTaskRequest` e `TaskResource`.

Justificativa. O Request normaliza e valida, o controller orquestra, o model persiste e o Resource fixa o contrato. Cada parte pode ser lida e testada isoladamente.

Trade-off. Há mais arquivos do que na versão inicial, mas cada um possui uma responsabilidade reconhecível e nativa do framework.

Resultado. Controller com apenas `index`, `store` e `destroy`; respostas sem timestamps, campos internos ou wrapper `data`.

5.4 Ausência de camada Service/Repository no backend

Problema. Era necessário decidir se Eloquent deveria ser ocultado por abstrações adicionais.

Alternativas. Repository interfaces, service de domínio, DTOs; ou recursos nativos diretos.

Decisão. Não criar essas camadas.

Justificativa. Três operações simples não justificam indireção extra. Eloquent, Form Request, Resource e controller já separam as responsabilidades relevantes.

Trade-off. O controller conhece Eloquent. Se regras complexas ou múltiplas fontes de dados surgirem, uma camada de aplicação poderá ser extraída com evidência concreta.

Resultado. Menos boilerplate e fluxo fácil de rastrear sem sacrificar validação ou testabilidade.

5.5 Componentização Angular

Problema. O componente raiz fazia comunicação, estado, formulário, lista e estilos.

Alternativas. Manter componente único; criar muitos componentes granulares; separar por responsabilidade de tela.

Decisão. `AppComponent` como shell, `TaskPageComponent` como container, `TaskFormComponent`, `TaskListComponent`, modelos e `TaskService`.

Justificativa. A divisão acompanha fronteiras reais: entrada, apresentação da coleção, coordenação de estado e infraestrutura HTTP.

Trade-off. O container ainda coordena várias flags, mas o volume é pequeno e explícito. Uma store global seria desproporcional.

Resultado. Componentes standalone, templates menores, service reutilizável e specs focados.

5.6 Reactive Forms

Problema. O formulário anterior não representava as regras do backend nem oferecia um fluxo de submit testável.

Alternativas. Template-driven forms; manipulação manual do input; Reactive Forms.

Decisão. `FormGroup` e `FormControl` não anulável com `required`, `maxLength` e validação de whitespace.

Justificativa. Reactive Forms torna estado e regras explícitos, integra o submit semântico e facilita teste de validade, trim e reset.

Trade-off. Há um pouco mais de código TypeScript do que em formulário template-driven, compensado por previsibilidade.

Resultado. Submit nativo, botão coerente, campo preservado em falha e reset apenas após 201.

5.7 Estado local em vez de NgRx

Problema. Era necessário controlar loading, envio, remoções e erros concorrentes.

Alternativas. NgRx/store global; signals globais; estado local no container.

Decisão. Propriedades tipadas locais no `TaskPageComponent` e RxJS somente nas chamadas assíncronas.

Justificativa. O estado pertence a uma única página e não é compartilhado. A solução local é mais legível para o tamanho da aplicação.

Trade-off. Se múltiplas rotas passarem a compartilhar tarefas, será necessário reavaliar o escopo do estado.

Resultado. Estados independentes para carregar, enviar e remover, sem biblioteca adicional.

5.8 Prevenção de duplicidade

Problema. Títulos equivalentes por `trim` e caixa podiam coexistir; uma consulta prévia isolada não protegeria duas requisições simultâneas.

Alternativas. Validar somente no cliente; consultar apenas no backend; constraint única sobre uma chave normalizada.

Decisão. Combinar validação local, consulta amigável no backend e índice unique em `normalized_title`.

Justificativa. O cliente melhora a experiência; o backend continua fonte da regra; o banco resolve race conditions. `mb_strtolower(trim(...), 'UTF-8')` centraliza a normalização.

Trade-off. A coluna adicional é interna e precisa de backfill em migration. Acentos não são normalizados por decisão de escopo.

Resultado. Duplicatas retornam 422, a grafia aparada é preservada e a violação concorrente específica não vira 500.

5.9 Docker por finalidade

Problema. Uma única imagem tenderia a acumular Xdebug e Chromium, aumentando peso e superfície do ambiente normal.

Alternativas. Uma imagem completa; Dockerfiles separados; estágios e Compose complementares.

Decisão. Compose principal para desenvolvimento, override para debug e serviço/estágio isolado para testes.

Justificativa. Mantém comandos claros e reaproveita camadas, sem colocar ferramentas pesadas onde não são necessárias.

Trade-off. Existem três combinações de Compose para documentar e manter.

Resultado. Backend normal sem Xdebug, debug com Xdebug 3.5.3 e frontend normal sem Chromium, enquanto `angular-test` possui Chromium 150.

5.10 TLS e CA externa

Problema. A inspeção HTTPS do ambiente causava `SELF_SIGNED_CERT_IN_CHAIN` durante `npm ci` no Docker.

Alternativas. Desabilitar validação TLS; usar HTTP; versionar CA; montar a CA externamente.

Decisão. Manter `strict-ssl=true` e fornecer a CA local como secret/mount fora do Git.

Justificativa. A cadeia passa a ser confiável sem enfraquecer transporte e sem expor certificado corporativo no repositório ou em camada final inadequada.

Trade-off. O desenvolvedor precisa preparar `.docker/certs/npm-ca.pem` no primeiro uso.

Resultado. Builds reproduzíveis no ambiente inspecionado, com `NODE_EXTRA_CA_CERTS` e `NPM_CONFIG_CAFILE` apontando para `/run/secrets/npm-ca.pem`.

6. Melhorias no backend

6.1 Estrutura final

O backend usa uma organização Laravel convencional:

- `Task` define fillable seguro, default e cast booleano;
- migrations criam `tasks` e adicionam a chave normalizada única;
- `TaskController` implementa somente listagem, criação e remoção;
- `StoreTaskRequest` executa `trim`, regras `required|string|max:255` e mensagem de duplicidade;
- `TaskResource` expõe apenas `id`, `title` e `completed`;
- `TaskSeeder` fornece três tarefas determinísticas;
- `TaskFactory` prepara estado isolado nos testes;
- CORS usa middleware nativo e origem configurável por `CORS_ALLOWED_ORIGINS`.

O usuário demonstrativo do scaffolding foi removido do `DatabaseSeeder`, pois não participa da checklist. O scaffolding Laravel não relacionado foi preservado para evitar limpeza sem valor funcional.

6.2 Validação e erros

Título ausente, vazio, composto apenas por espaços, array, objeto ou acima de 255 caracteres retorna `422`. A normalização acontece antes da validação e o mutator preserva o título aparado. A criação sempre começa incompleta.

A remoção usa route model binding. Um ID existente retorna `204` sem corpo; um ID inexistente retorna `404` nativo.

A violação de unicidade é tratada somente quando o SQLSTATE é `23000` e a mensagem identifica `tasks.normalized_title`. Outros `QueryException` são relançados, evitando mascarar falhas internas.

6.3 Contrato final

Método	Caminho	Entrada	Saída	Status
GET	/tarefas	-	array sem wrapper, ordenado por ID	200
POST	/tarefas	{ "title": "..." }	objeto sem wrapper	201, 422
DELETE	/tarefas/{task}	-	corpo vazio	204, 404

Objeto público:

```
# json
{
  "id": 4,
  "title": "Nova tarefa",
  "completed": false
}
```

`normalized_title`, `created_at` e `updated_at` não aparecem na API.

7. Melhorias no frontend

O `AppComponent` tornou-se um shell que apenas renderiza a página. O `TaskPageComponent` carrega, ordena e atualiza a coleção após confirmações reais. O `TaskService` centraliza `/tarefas` e retorna `Observable<Task[]>`, `Observable<Task>` e `Observable<void>` sem `any` ou `fallback`.

O formulário usa `[formGroup]`, `formControlName`, `<form (ngSubmit)>` e botão `type="submit"`. O payload usa `trim`; `whitespace` e mais de 255 caracteres são inválidos. `isSubmitting` bloqueia envios concorrentes. O reset só ocorre no callback de sucesso. Se o backend falha, o texto permanece e a mensagem segura é associada ao campo.

A página apresenta loading inicial, estado vazio, retry, erro de criação e erro de remoção. A lista só adiciona o objeto retornado pelo POST e só remove após DELETE bem-sucedido. Uma falha de remoção conserva o item. IDs em remoção são mantidos em `ReadonlySet`, permitindo bloquear apenas o botão correto.

A validação local de duplicidade aplica `trim().toLowerCase()`, coerente com a regra funcional do backend. Ela evita POST quando a lista local já contém o título, mas o tratamento de `422 errors.title` permanece obrigatório para lista desatualizada ou concorrência.

Acessibilidade e responsividade incluem:

- landmarks `main`, `section` e `header`;
- label associado ao input;
- `aria-invalid`, `aria-describedby`, `aria-live` e `aria-busy`;
- texto contextual no botão de remoção;
- foco visível e navegação por teclado;
- estados não dependentes apenas de cor;
- layout mobile first, controles empilhados em telas estreitas;
- `overflow-wrap:anywhere` para títulos longos;
- ausência de estilos inline e de Router sem uso.

Subscriptions são encerradas com `takeUntilDestroyed`. Não há atualização otimista, dados offline, IDs aleatórios ou URL absoluta do backend.

8. Estratégia de testes

8.1 Backend

Os testes Feature registram o contrato e as correções intencionais. `RefreshDatabase` usa SQLite em memória, eliminando dependência do banco local e ordem entre testes. A suíte cobre forma e tipos de GET, ordenação, booleano, criação, trim, persistência, validações 422, remoção 204 e 404, middleware `api`, CORS, duplicidade, não exposição do campo interno, constraint direta e seeder.

O arquivo foi renomeado de `TaskApiCharacterizationTest.php` para `TaskApiTest.php`, pois agora descreve o contrato final, não apenas o comportamento legado.

8.2 Frontend

Os specs usam `TestBed`, `provideHttpClientTesting`, spies e Observables controlados. Não realizam HTTP real. Eles cobrem métodos, URLs, payloads, retornos e propagação de erros do service; validade, whitespace, limite, submit, trim, bloqueio e ARIA do formulário; loading, retry, criação, reset, preservação em falha e remoção na página; duplicidade local e 422; além de lista, estado vazio e bloqueio individual.

O runner `angular-test` instala Chromium somente no estágio de testes. O launcher `ChromeHeadlessDocker` usa `--no-sandbox` e `--disable-dev-shm-usage` dentro do container isolado.

8.3 Resultado medido

- backend: 28 de 28 testes aprovados, 106 assertions;
- frontend: 24 de 24 specs aprovados no Chromium;
- 83 chamadas estáticas de assertion nos arquivos `.spec.ts`;
- nenhum teste skipped ou desabilitado;
- cobertura Angular gerada com Istanbul/Karma.

9. Uso de Inteligência Artificial

A Inteligência Artificial foi utilizada como ferramenta de apoio à análise, planejamento, implementação e revisão. As decisões arquiteturais, a validação dos resultados e a responsabilidade final pela solução permaneceram humanas.

O uso foi explícito e delimitado. A IA apoiou:

- leitura e categorização inicial da estrutura;
- identificação de responsabilidades misturadas e riscos de integridade;
- construção do plano incremental e dos critérios de conclusão;
- sugestões proporcionais de arquitetura Angular e Laravel;
- geração e revisão inicial de testes de contrato e componente;
- investigação dos mounts, estágios e cache Docker;
- diagnóstico do certificado TLS e preservação de `strict-ssl`;
- configuração inicial de Xdebug, source maps e VS Code;
- revisão de semântica, acessibilidade e responsividade;
- verificação de padrões inseguros e documentação técnica;
- organização desta auditoria e do relatório.

Permaneceram sob responsabilidade humana:

- escolha final entre alternativas e aceite de trade-offs;
- revisão de todo o código sugerido;
- execução dos comandos e leitura das saídas reais;
- validação funcional dos endpoints e interface;
- observação da regressão do formulário que não disparava POST;
- confirmação da causa antes da correção;
- avaliação de proporcionalidade e rejeição de abstrações desnecessárias;
- rejeição de soluções inseguras como desabilitar TLS;
- confirmação de migrations, rollback, testes, cobertura e builds;
- responsabilidade sobre conteúdo, entrega e limitações declaradas.

A IA não é apresentada como autora autônoma. Ela operou como um segundo par de análise, acelerando exploração e revisão, enquanto evidência executável determinou se cada sugestão seria mantida.

10. Dosagem de IA

Etapa	Contribuição da IA	Validação humana	Resultado
Diagnóstico	Categorização de problemas e riscos	Leitura e confirmação arquivo a arquivo	Plano incremental controlado
Ambiente Docker	Sugestões de estágios, volumes e endpoints	Builds e comandos no Docker Desktop	Ambiente normal reproduzível
TLS	Hipóteses sobre inspeção HTTPS e CA externa	Verificação de <code>strict-ssl</code> e CA	TLS preservado sem CA versionada
Debug	Configuração inicial de Xdebug e launchers	<code>php -ri xdebug</code> e path mappings	Debug isolado
Backend	Estrutura controller/request/resource/model	Testes, migrations, rollback e Pint	API stateless com Eloquent
Frontend	Separação shell/container/form/list/service	Build, specs e fluxo	UI tipada e sem falso sucesso
Duplicidade	Estratégia em três níveis	Constraint, 422 e testes	Integridade protegida no banco
Testes	Geração inicial de cenários e mocks	Execução em PHPUnit e Chromium	28 testes e 24 specs
Documentação	Estrutura, síntese e revisão textual	Conferência com resultados reais	README e relatório rastreáveis

As tarefas foram fornecidas em prompts delimitados, cada um com proibições, escopo, comandos obrigatórios e critérios de conclusão. Esse desenho controlou a dosagem: a IA não recebeu autorização genérica para reescrever o produto; recebeu problemas pequenos, evidências esperadas e limites explícitos.

Uma sugestão só foi aceita quando coerente com três filtros: atende ao contrato, é proporcional ao domínio e pode ser validada. Esse processo evitou upgrades, frameworks adicionais, observabilidade pesada, autenticação, repository pattern e outras expansões sem requisito.

11. Impacto técnico e de negócio

11.1 Confiabilidade

A interface agora reflete o servidor. Uma criação falha não gera tarefa local; uma remoção falha não apaga o item visual; e duplicidade concorrente encontra uma constraint definitiva. Isso reduz discrepância de dados e melhora confiança do usuário.

11.2 Integridade de dados

SQLite e migrations substituem manipulação integral de JSON. IDs, tipos, constraints e rollback são gerenciados pelo banco. A coluna normalizada impede títulos equivalentes mesmo sob duas requisições simultâneas.

11.3 Onboarding e produtividade

Comandos Docker documentados reproduzem desenvolvimento, debug e testes. Volumes nomeados evitam reinstalar dependências a cada restart. A separação entre ambientes reduz diagnósticos causados pela máquina local e permite que uma nova pessoa alcance o baseline mais cedo.

11.4 Custo de manutenção

Arquivos com responsabilidades claras e testes de contrato reduzem o contexto necessário para mudanças. Uma alteração no formulário pode ser testada sem HTTP real; uma mudança de serialização está concentrada no Resource; uma regra de entrada vive no Request.

11.5 Experiência do usuário

Loading, estado vazio, retry e erros específicos tornam o comportamento compreensível. Layout mobile first, quebra de títulos e controles acessíveis ampliam a utilidade em telas pequenas. Preservar o texto em falha evita retrabalho.

11.6 Evolução segura

O projeto não tenta antecipar todas as necessidades, mas cria pontos claros de extensão. O contrato está testado, a persistência possui migrations e o frontend não depende de URLs absolutas. Isso reduz regressões quando funcionalidades futuras forem priorizadas.

12. Indicadores de sucesso

12.1 Cobertura de testes

A cobertura representa a proporção de elementos instrumentados executados pela suíte:

text

Cobertura = elementos executados pelos testes / total de elementos instrumentados × 100

Métrica Angular	Executado / total	Cobertura
Statements	77 / 82	93,90%
Branches	17 / 21	80,95%
Functions	31 / 31	100,00%
Lines	73 / 78	93,58%

A menor métrica é branches, ainda em 80,95%. Isso é coerente com caminhos defensivos menos frequentes no parser de erro e guard clauses. Não foi imposto 100%, pois o objetivo é proteger comportamento crítico, não maximizar um número isolado.

12.2 Taxa de testes aprovados

text

Taxa de aprovação = testes aprovados / testes executados × 100

Suíte	Aprovados	Executados	Taxa
Backend PHPUnit	28	28	100%

Suíte	Aprovados	Executados	Taxa
Frontend Karma/Jasmine	24	24	100%

Esses valores medem a execução final local em Docker, não desempenho em produção.

12.3 Confiabilidade do fluxo crítico

Fluxo	Evidência automatizada	Evidência de integração
Listar	contrato, tipos, ordenação e service GET	backend e proxy retornaram 200
Criar	POST, payload, trim, estado e reset	criação temporária retornou 201
Validar	entradas inválidas e FormControl	duplicidade direta retornou 422
Bloquear duplicidade	cliente, Request e unique constraint	mensagem presente em <code>errors.title</code>
Remover	sucesso, falha e preservação visual	DELETE 204; repetição 404

A duplicidade local sem POST foi executada como spec real no Chromium. A automação visual do navegador integrada ao ambiente de auditoria não abriu por restrição de ACL; essa limitação é declarada, sem converter teste unitário em alegação de validação manual.

12.4 Indicadores recomendados para produção

Os indicadores abaixo não foram medidos neste desafio e são recomendações para um ambiente publicado:

- taxa de respostas HTTP 5xx e 4xx por endpoint;
- p50, p95 e p99 de latência;
- volume e motivo das falhas 422;
- tentativas de duplicidade por período;
- disponibilidade da API e do frontend;
- tempo médio para detectar e resolver falhas;
- regressões identificadas antes e depois do deploy;
- taxa de sucesso dos fluxos listar, criar e remover;
- erros de JavaScript por sessão e tipo de dispositivo.

Métricas técnicas devem ser correlacionadas com experiência: aumento de 422 de duplicidade pode indicar proteção funcionando, mas também necessidade de melhorar comunicação ou atualização da lista.

13. Resultados finais

13.1 Backend

- `migrate:fresh --seed`: aprovado;
- `migrate:rollback`: aprovado para todas as migrations;
- `route:list --path=tarefas -v`: três rotas com middleware `api`;
- PHPUnit: 28 testes e 106 assertions, todos aprovados em 3,14 s;
- Pint: 37 arquivos aprovados;
- PHP 8.3.32, Laravel 11.45.1 e SQLite 3.46.1;
- `normalized_title` e timestamps ausentes das respostas;
- usuário demonstrativo removido do seeder.

13.2 Frontend

- 24 specs aprovados em Chromium 150;
- 83 chamadas estáticas de assertion;
- TypeScript dos specs aprovado sem emissão;
- build de produção aprovado em 5,170 s;
- bundle inicial de 210,78 kB, estimativa transferida de 60,34 kB;
- Angular 17.3.12, TypeScript 5.4.5 e Node 20.20.2;
- cobertura gerada e não versionada.

13.3 Docker e segurança

- `docker compose config`: aprovado;
- `override debug`: aprovado;
- `override test`: aprovado;
- build normal, debug e test: aprovados;
- imagem backend normal sem Xdebug;
- imagem debug com Xdebug 3.5.3, client port 9003 e host `host.docker.internal`;
- porta 9003 não publicada;
- imagem Angular normal sem Chromium;
- runner test com `/usr/bin/chromium`;
- `strict-ssl=true` e CA em secret externo;
- bind mounts alinhados e dependências em volumes nomeados.

13.4 Integração HTTP

A sonda final enviou `Accept: application/json`, como o cliente Angular:

Verificação	Resultado
Frontend	200
GET backend	200
GET pelo proxy	200
POST único pelo proxy	201
Duplicata direta	422
Campo de erro	Já existe uma tarefa com esse título.
DELETE pelo proxy	204
DELETE repetido	404
Tarefa temporária	removida

Os logs mostraram os servidores inicializados e as requisições, sem stack trace ou erro inesperado. O aviso do `ng serve` sobre uso exclusivo para desenvolvimento é esperado e está alinhado à finalidade do serviço.

14. Limitações e melhorias futuras

A entrega permanece intencionalmente pequena. Limitações conhecidas:

- não há autenticação ou autorização;

- tarefas não podem ser editadas ou marcadas como concluídas pela interface;
- a listagem não possui paginação;
- não houve deploy real ou validação de infraestrutura de produção;
- o proxy Angular é exclusivo do desenvolvimento;
- SQLite é adequado ao case, mas um banco servidor pode ser necessário para escala e concorrência maiores;
- não há pipeline CI/CD;
- não há suíte E2E completa dirigindo um navegador contra backend real;
- a inspeção visual interativa final ficou limitada pela automação de navegador indisponível por ACL, embora responsividade e fluxos estejam cobertos por implementação, specs e validações HTTP.

Melhorias futuras devem responder a necessidade real:

- 1 edição e conclusão de tarefas;
- 2 autenticação quando houver dados por usuário;
- 3 paginação e filtros quando o volume justificar;
- 4 E2E para os fluxos principais;
- 5 CI/CD com PHPUnit, Pint, Karma, TypeScript e builds;
- 6 imagens de produção e proxy reverso;
- 7 métricas e logs estruturados para os indicadores recomendados.

15. Conclusão

A refatoração transformou uma checklist frágil em uma base simples, verificável e coerente com seu tamanho. O ganho principal não foi adicionar funcionalidades, mas tornar o comportamento confiável: dados válidos persistem, erros permanecem erros, contratos são explícitos e concorrência não viola unicidade.

A solução equilibra manutenção e proporcionalidade. Laravel fornece separação com controller, request, resource e Eloquent; Angular fornece componentes standalone, Reactive Forms e estado local; Docker fornece ambientes por finalidade. Camadas e ferramentas sem necessidade foram conscientemente evitadas.

O uso de Inteligência Artificial foi transparente e controlado. Ela acelerou análise e produção inicial, mas evidências executáveis, revisão crítica e decisão humana determinaram o resultado. Essa combinação preservou o olhar de especialista: entender o impacto para usuário e negócio, escolher trade-offs adequados e declarar honestamente o que foi ou não validado.

O projeto termina com testes aprovados, cobertura relevante, documentação operacional e um caminho claro para evolução. A entrega prioriza confiabilidade, simplicidade, manutenibilidade e comunicação técnica responsável.